

Licenciatura em Telecomunicações e Informática
Arquitetura e Tecnologia de Computadores

1st Semester 2024/2025

Transmissão de dados UART/USART

Tiago Gomes

Embedded Systems Research Group (ESRG)

Gab. Ed2-1.56/1.57

mr.gomes@dei.uminho.pt



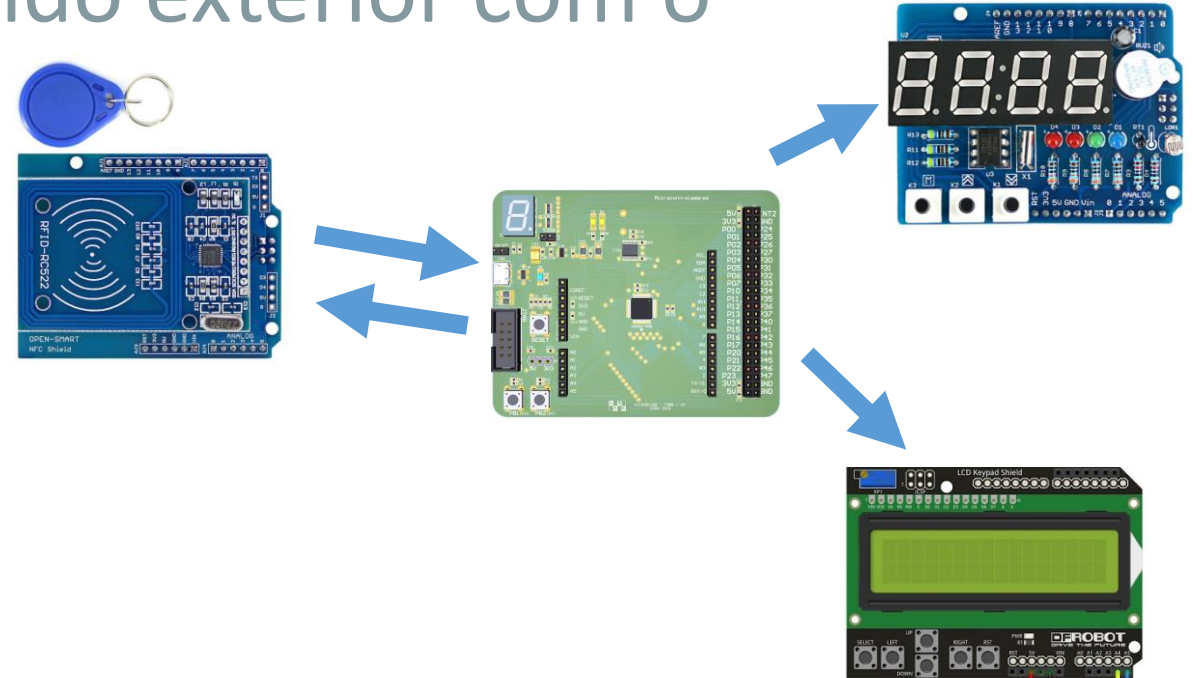
AGENDA

- 01 Transmissão de Dados
- 02 Periféricos UART no 8051F388
- 03 Exemplos com UART (Linguagem C)

Transmissão de dados

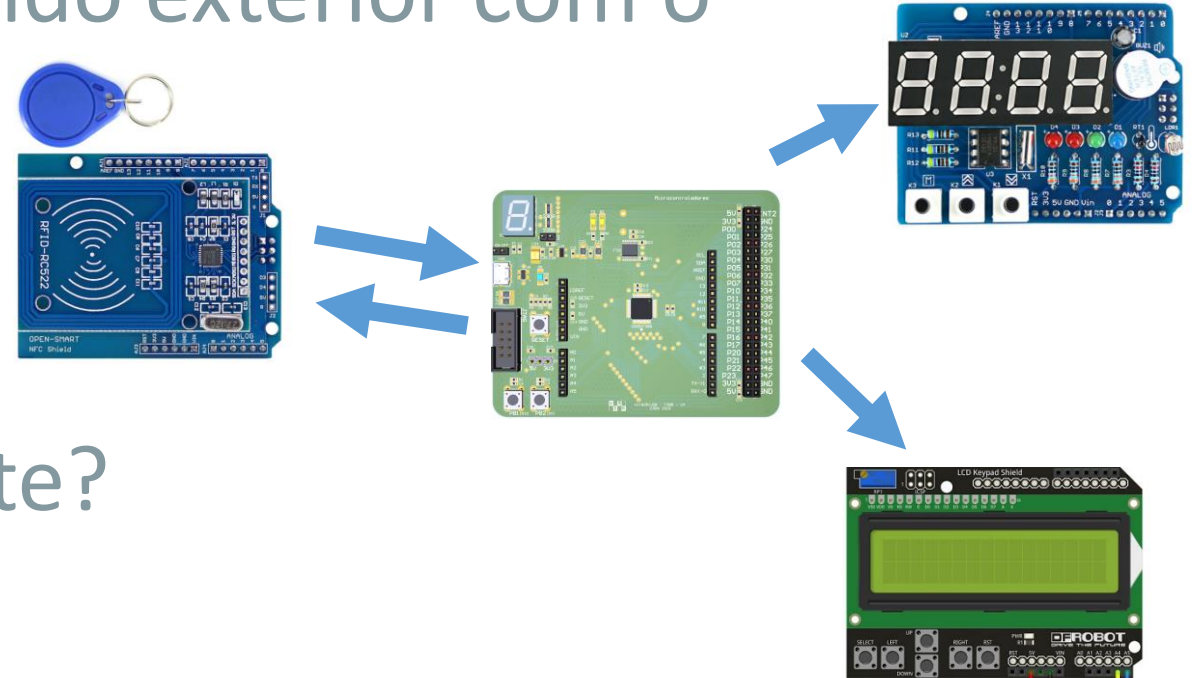
Transmissão de Dados

- Como fazer interface do mundo exterior com o Microcontrolador?
- I/Os analógicos...
- I/Os digitais...



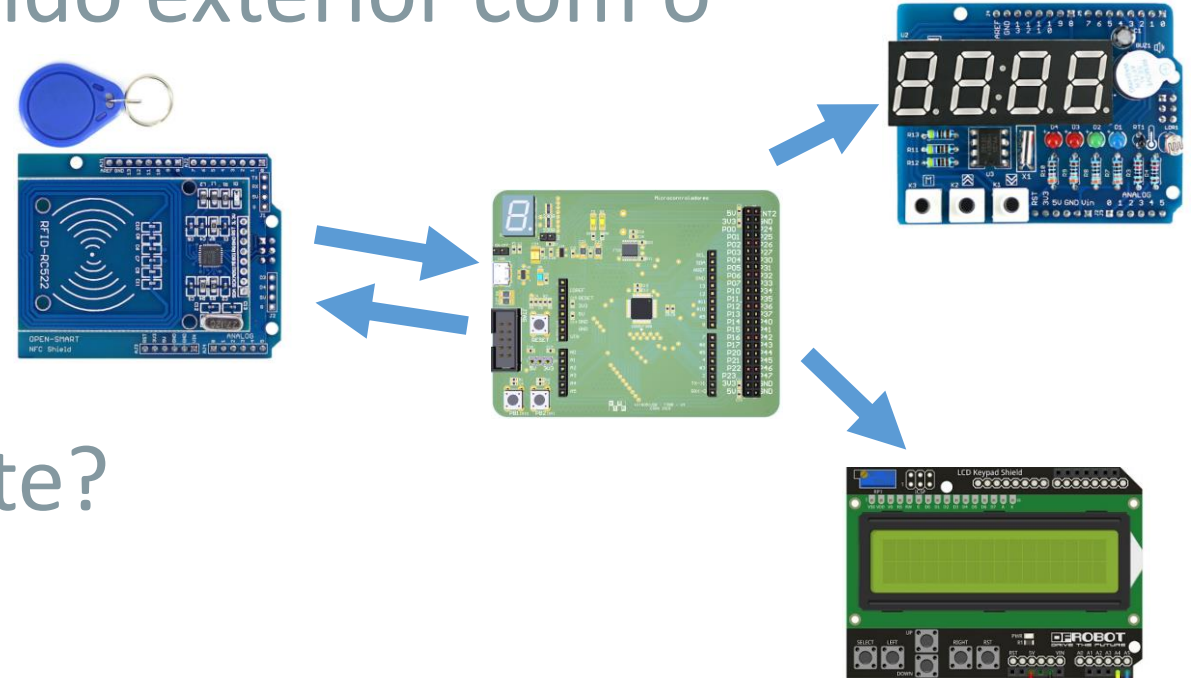
Transmissão de Dados

- Como fazer interface do mundo exterior com o Microcontrolador?
 - I/Os analógicos...
 - I/Os digitais...
- Como comunicar digitalmente?



Transmissão de Dados

- Como fazer interface do mundo exterior com o Microcontrolador?
 - I/Os analógicos...
 - I/Os digitais...
- Como comunicar digitalmente?
- Como transmitir um ou mais bytes através de um canal de comunicações (um ou mais fios)?



Métodos de Transmissão

- Enviar a string 'LETI'
 - 'L': ?
 - 'E': ?
 - 'T': ?
 - 'I': ?

Métodos de Transmissão

- Enviar a string 'LETI'
 - 'L': ?
 - 'E': ?
 - 'T': ?
 - 'I': ?

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

Métodos de Transmissão

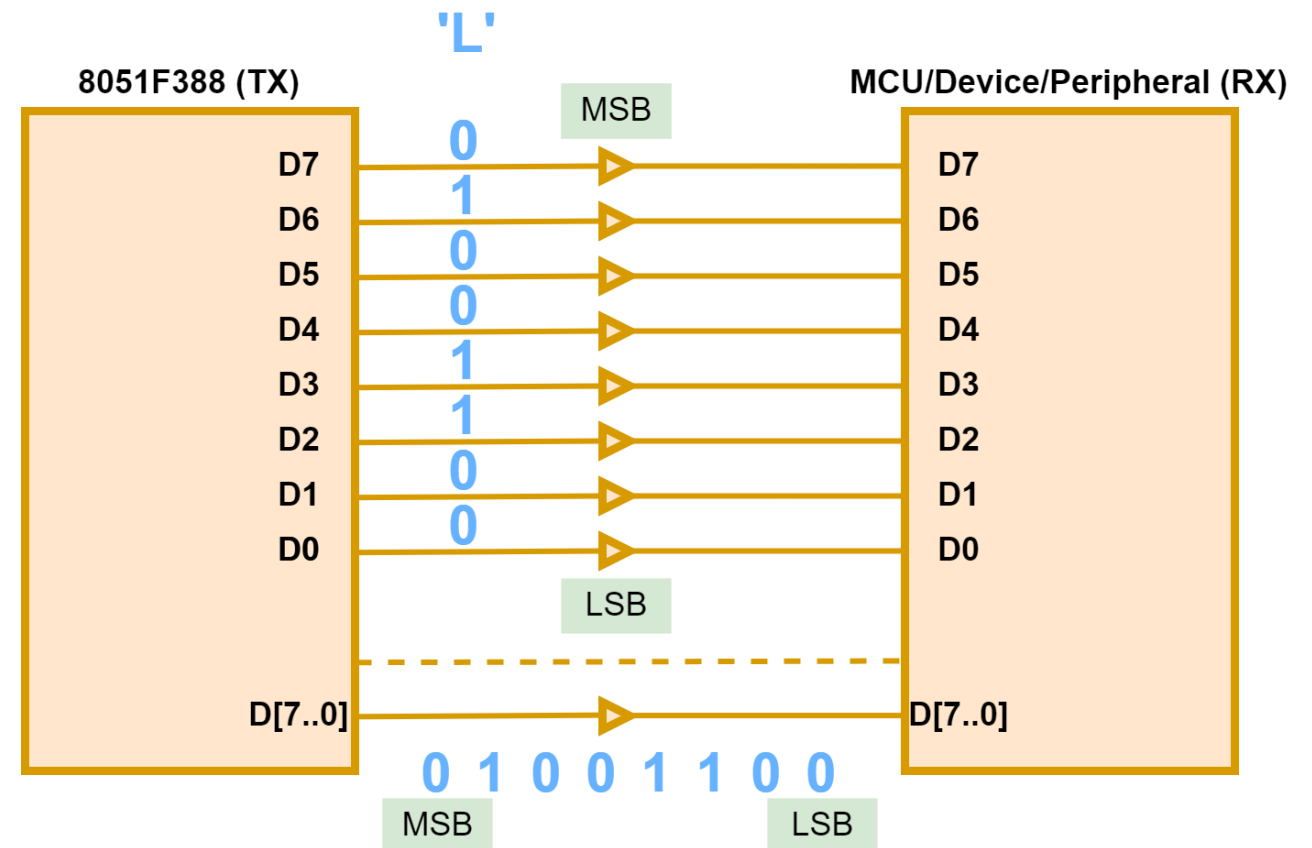
- Enviar a string 'LETI'
 - 'L': 76d; 4Ch; 01001100b
 - 'E': 69d; 45h; 01000101b
 - 'T': 84d; 54h; 01010100b
 - 'I': 73d; 49h; 01001001b

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

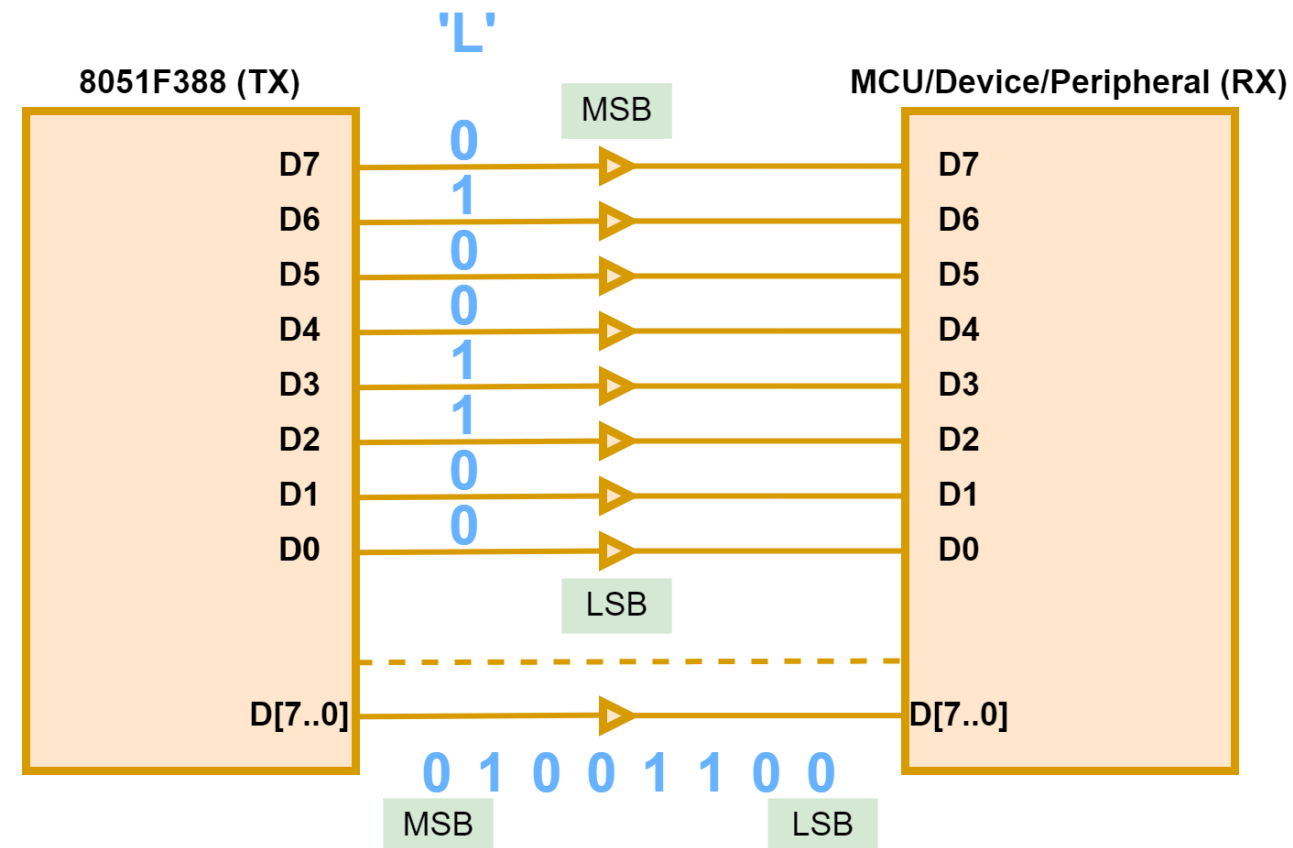
Métodos de Transmissão

- Série vs. Paralela
- 'LETI'
 - 76, 69, 84, 73
 - 0x4C, 0x45, 0x54, 0x49
 - 01001100b, (...)

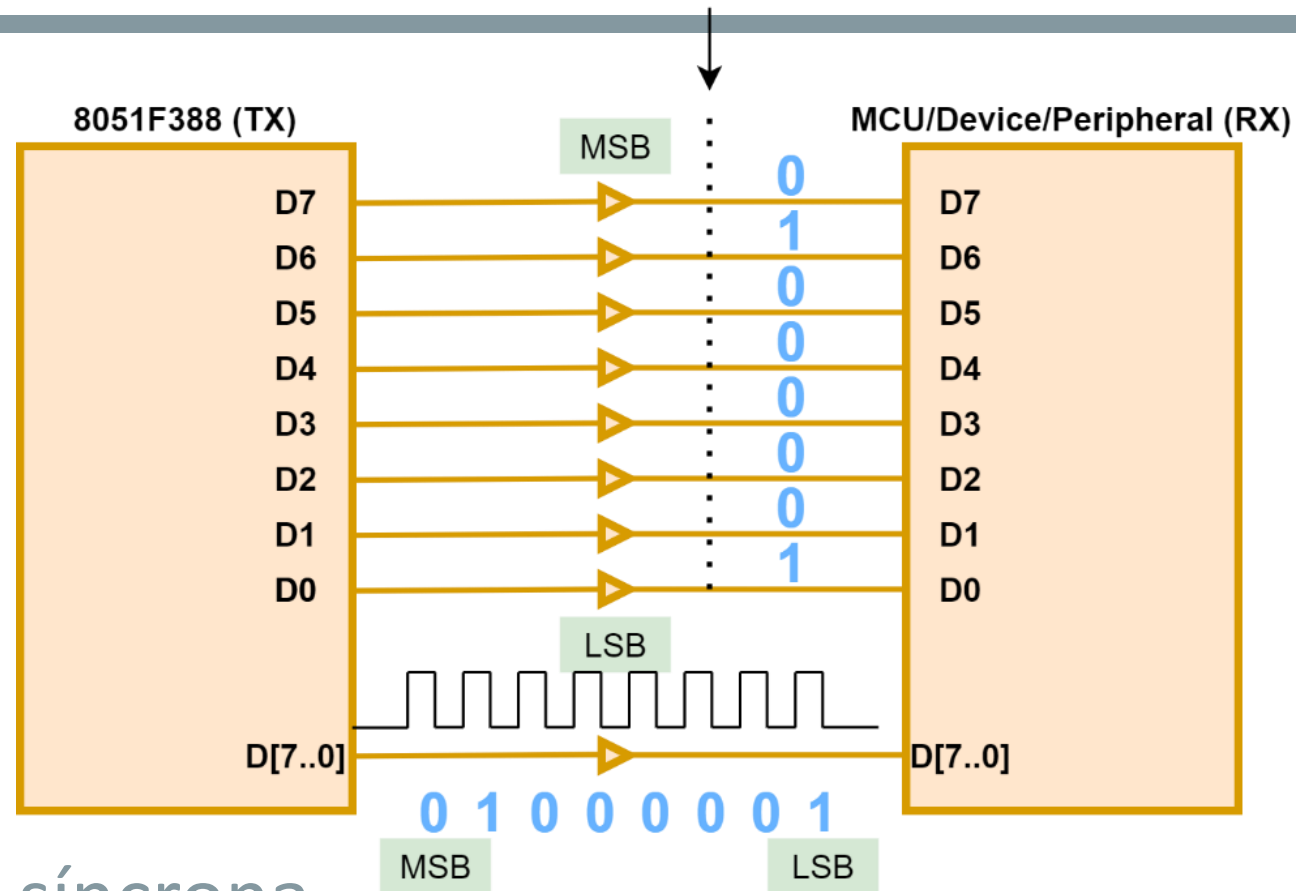


Métodos de Transmissão

- Série
 - Menos hardware necessário (fios/Portos I/O)
 - Mais “lenta”? Transmite um bit de cada vez...
- Paralela
 - Necessário mais hardware (fios/Portos I/O)
 - Mais “rápida”? Transmite mais bits em simultâneo...

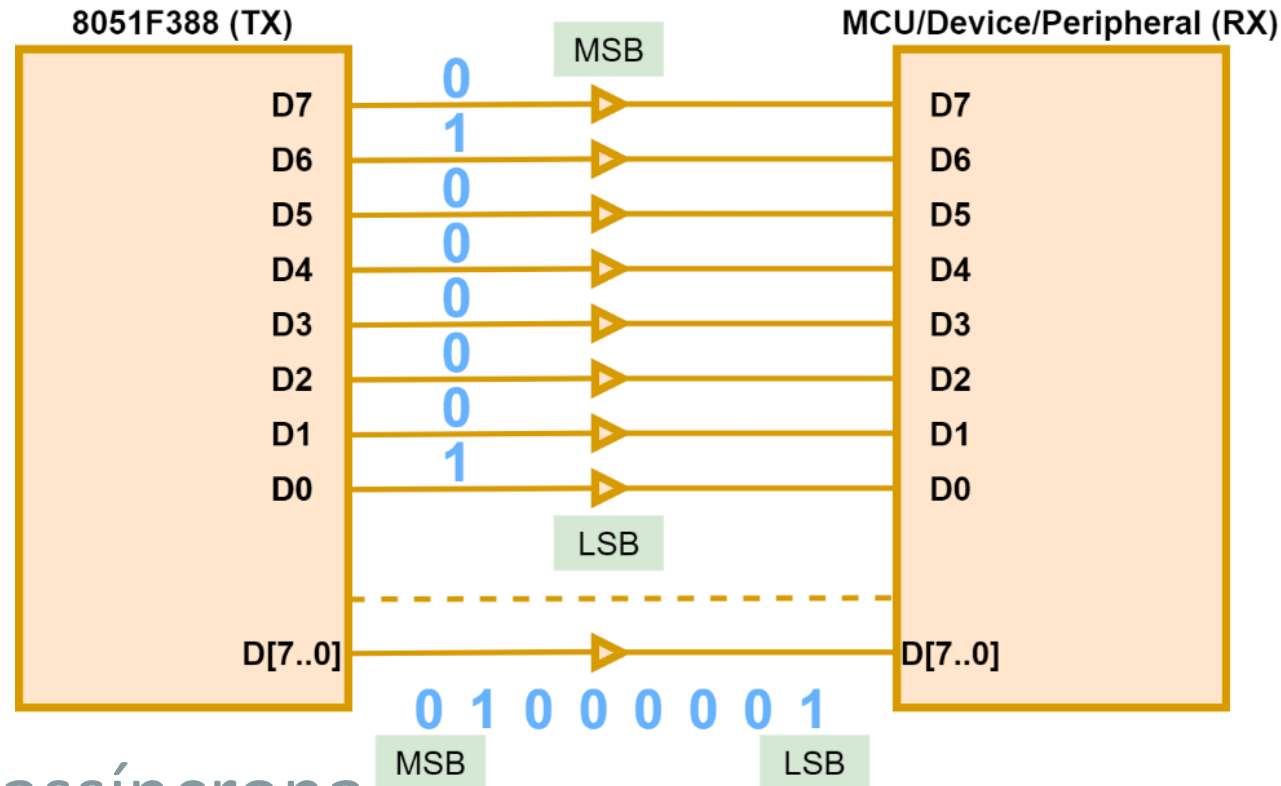


Métodos de Transmissão



- Transmissão síncrona
 - Necessário um sinal extra, que indica que um bloco de dados (1 bit, 4 bits, 1 byte) está pronto para ser lido;
 - Geralmente usa-se um sinal de clock (CLK).

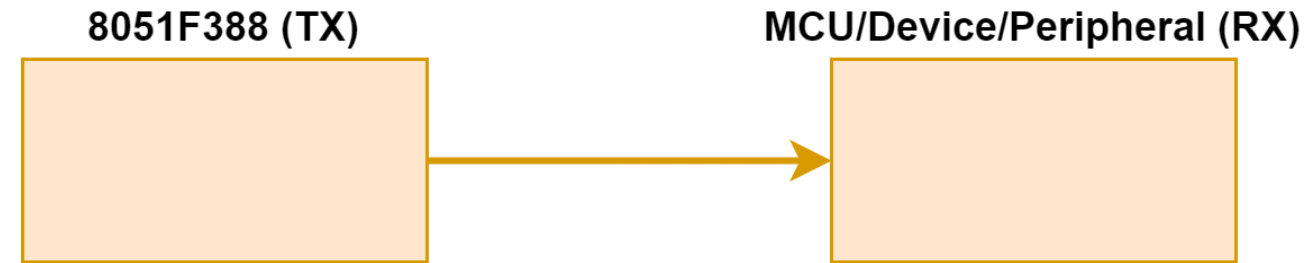
Métodos de Transmissão



- Transmissão assíncrona
 - O sinal de sincronismo é extraído pela análise do sinal que transporta os dados.
 - Temporal?
 - Técnicas de modelação/codificação?

Comunicação série

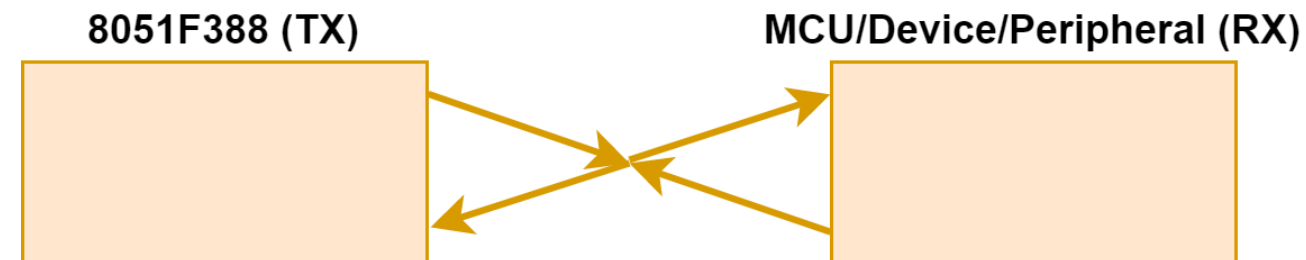
- Tipos de comunicação
 - Simplex: Comunicação é feita num único sentido;



Comunicação série

- Tipos de comunicação

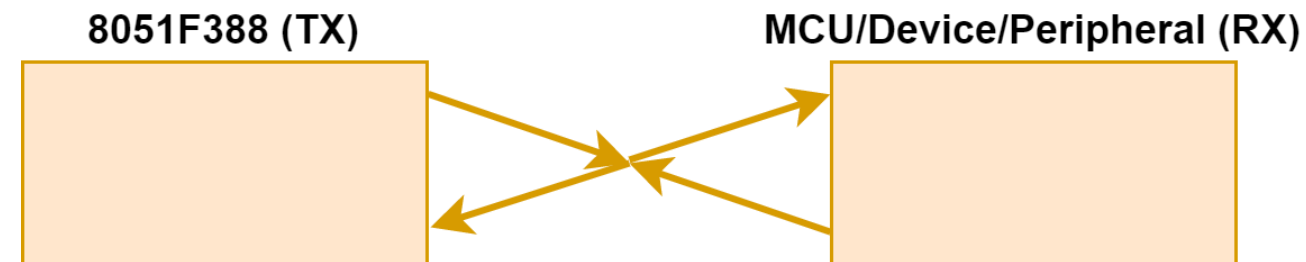
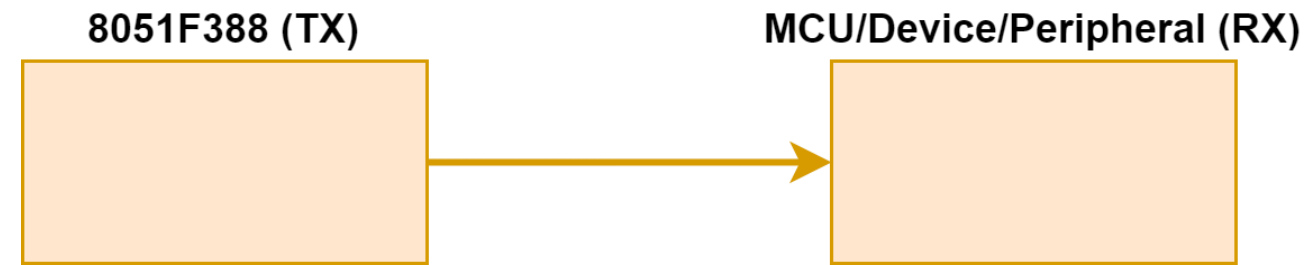
- Simplex: Comunicação é feita num único sentido;
- Half-Duplex: Comunicação efetuada nos dois sentidos, mas apenas um sentido de cada vez;



Comunicação série

- Tipos de comunicação

- Simplex: Comunicação é feita num único sentido;
- Half-Duplex: Comunicação efetuada nos dois sentidos, mas apenas um sentido de cada vez;
- Full-Duplex: Comunicação nos dois sentidos e simultaneamente;



Comunicação série

- Exemplos de Protocolos
 - Serial Peripheral Interface (SPI)
 - Master Input Slave Output (MISO); Master Output Slave Input (MOSI); Clock (CLK); Chip Select (CS).
 - Inter-Integrated Circuit (I²C)
 - Serial CLock (SCL); Serial Data (SDA).
 - RS-232 (Recommended Standard 232)
 - Transmit (TX); Receive (RX); Clear to Send (CTS), Request to Send (RTS);

Comunicação série

- Utilização:

“In telecommunications, RS-232, Recommended Standard 232 [1] is a standard originally introduced in 1960 [2] for **serial communication transmission of data**. It formally defines signals connecting between a DTE (data terminal equipment) such as a computer terminal, and a DCE (data circuit-terminating equipment or data communication equipment), such as a modem. “



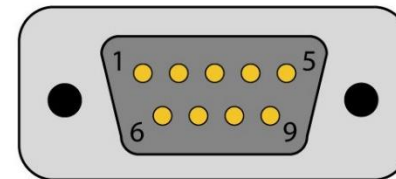
Connection	Modulation	Bitrate [kbit/s]	Year released
110 baud Bell 101 modem	FSK	0.1	1958
300 baud (Bell 103 or V.21)	FSK	0.3	1962
1200 modem (1200 baud) (Bell 202)	FSK	1.2	1976
1200 modem (600 baud) (Bell 212A or V.22)	QPSK	1.2	1980 ^[26] ^[27]
2400 modem (600 baud) (V.22bis)	QAM	2.4	1984 ^[26]
2400 modem (1200 baud) (V.26bis)	PSK	2.4	
4800 modem (1600 baud) (V.27ter)	PSK	4.8	^[28]
9600 modem (2400 baud) (V.32)	QAM	9.6	1984 ^[26]
14.4k modem (2400 baud) (V.32bis)	trellis	14.4	1991 ^[26]
19.2k modem (2400 baud) (V.32 "terbo")	trellis	19.2	1993 ^[26]
28.8k modem (3200 baud) (V.34)	trellis	28.8	1994 ^[26]
33.6k modem (3429 baud) (V.34)	trellis	33.6	1996 ^[29]
56k modem (8000/3429 baud) (V.90)	digital	56.0/33.6	1998 ^[26]
56k modem (8000/8000 baud) (V.92)	digital	56.0/48.0	2000 ^[26]
Bonding modem (two 56k modems) (V.92) ^[30]		112.0/96.0	
Hardware compression (variable) (V.90/V.42bis)		56.0–220.0	
Hardware compression (variable) (V.92/V.44)		56.0–320.0	
Server-side web compression (variable) (Netscape ISP)		100.0–1,000.0	

Comunicação série

- RS232 Interface física:
 - Conector de 25 e 9 pinos!
 - Sinais elétricos -12V e +12V representavam os valores lógicos “0” e “1”;
 - Atualmente usam-se níveis TTL para representar “0” = 0V e “1” = 3.3V; 5V



DB9M Connector

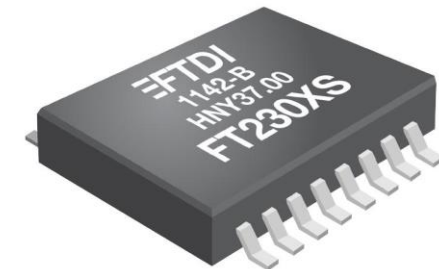
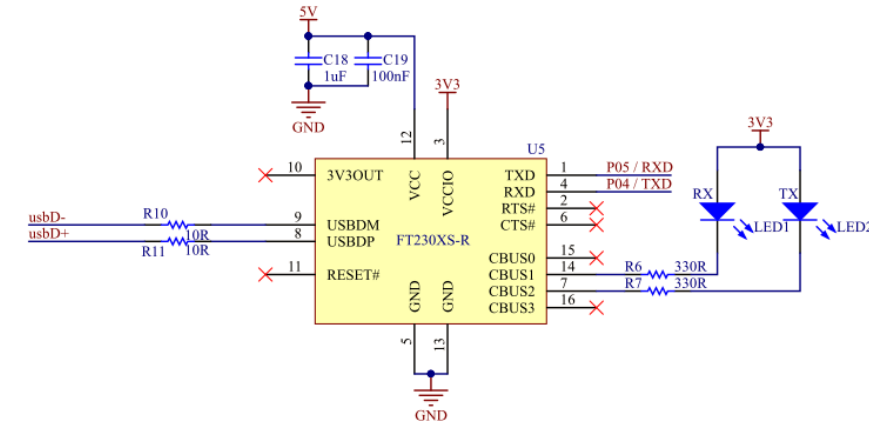


RS232 Pin Out

Pin #	Signal
1	DCD
2	RX
3	TX
4	DTR
5	GND
6	DSR
7	RTS
8	CTS
9	RI

Comunicação série

- RS232
 - Interface DB9 (conector) obsoleto
 - O KIT 8051F388 utilizada um conversor RS232 <-> USB, **FTDI (FT230XS)**
 - **FT230XS** fornece uma porta COM virtual, como se de uma física (ficha DB9) se tratasse
 - **FT230XS** está fisicamente ligado aos pinos TX (P0.4) e RX (P0.5) do 8051F388. Os pinos CTS e RTS do FTDI/uC não são utilizados!

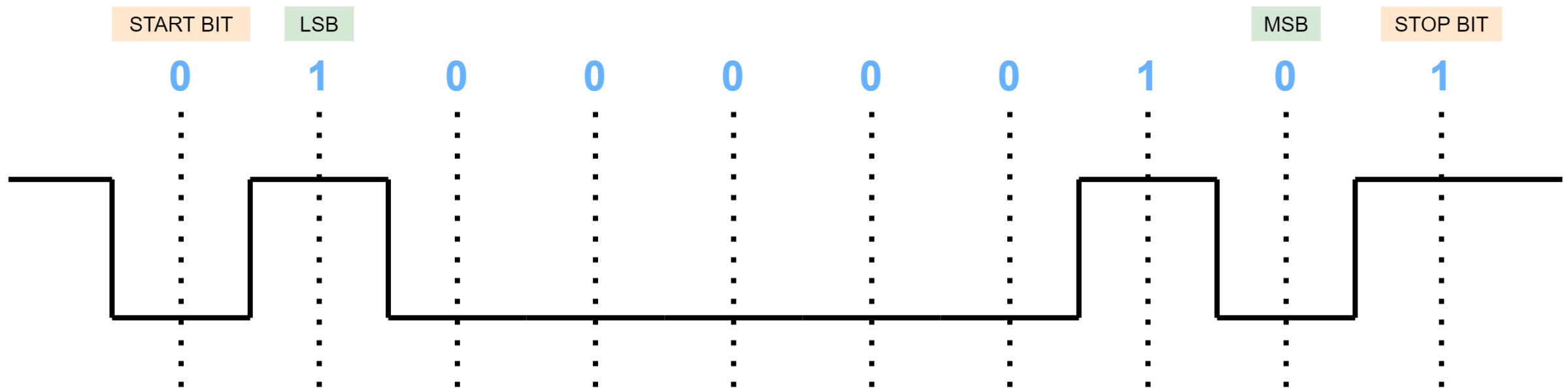


Comunicação série

- RS232
 - Protocolo assíncrono! É necessário extrair o sincronismo do sinal de dados.
 - A comunicação *full-duplex* pode ser conseguida apenas com os pinos TX e RX (CTS e RTS podem ser dispensados)
 - A taxa de transferência de dados é expressa em bits por segundo, *bits per second* (bps) e é designada de *baudrate*.
 - Valores típicos (standard) de *baudrate*:
 - 300; 600; 1200; 2400; 4800; 9600; 19200; 38400; 57600; 115200!

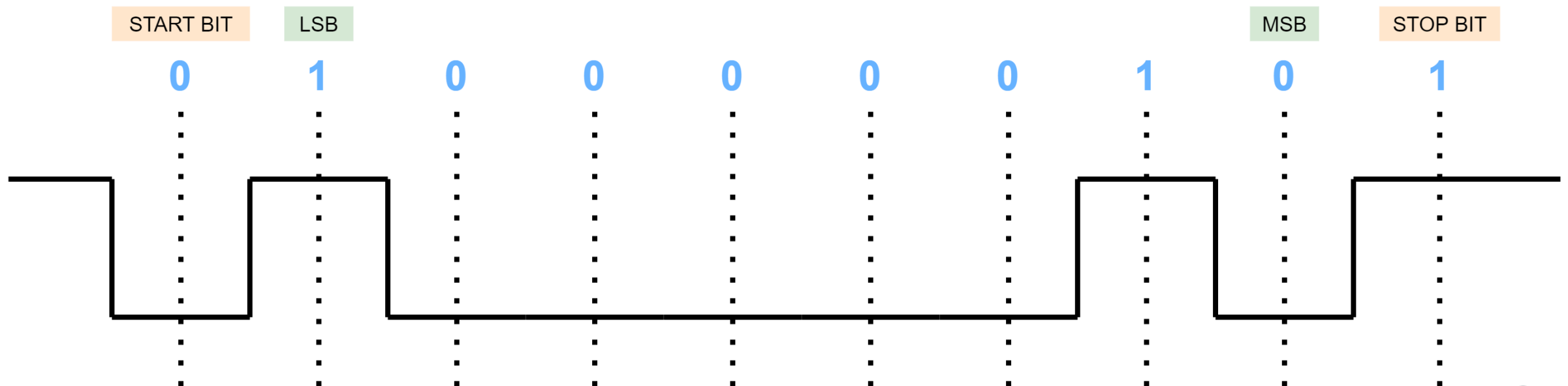
Comunicação série

- Trama RS232
 - “A” “0x41”, “0100 0001”
 - Start Bit, Indica o início da trama (byte)
 - Stop Bit, Indica fim de trama (byte)
 - Opcionalmente podem-se usar 2 Stop Bits e bit de paridade para controlo de erros.



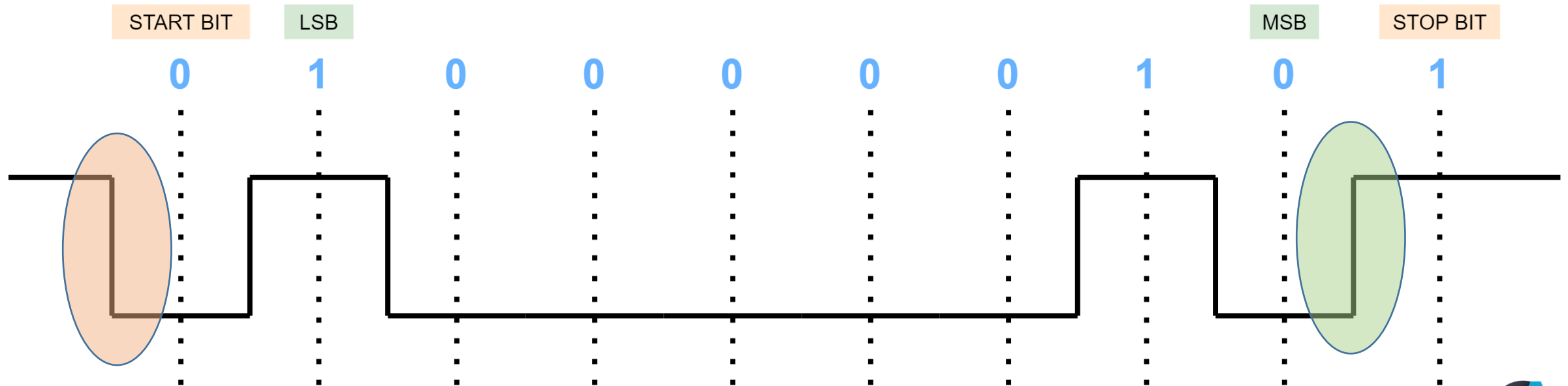
Comunicação série

- Como extrair o sincronismo?
 - É necessário primeiro o receptor negociar baudrate com o transmissor!
 - Amostra-se o Start Bit a uma frequência superior (8x, 16x, etc) do sinal amostrado para se conseguir determinar o “meio” do pulso;
 - Restantes bits são amostrados com intervalos conhecidos a partir do ponto de referência!



Comunicação série

- Para um *baudrate* de 9600 bps:
 - Aprox. um bit a cada 104us!
 - Detetado o start bit (transição 1->0 do sinal TX), esperam-se +-52 us, sendo os restantes bits amostrados a cada 104 us... até fim de trama (1 + 8 + 1 bits), sem considerar bit paridade!



UART no 8051F388

UART no 8051F388

- Controladores de comunicação UART
 - USART (Universal Synchronous-Asynchronous Receiver-Transmitter) – suportam comunicação síncrona e assíncrona
 - UART (Universal Asynchronous Receiver-Transmitter) – suporta comunicação assíncrona

UART no 8051F388

- Como implementar o protocolo série RS232 no 8051F388?
Usar técnicas de bit-banging?
- O C8051F388 tem hardware dedicado: UART0 e UART1!
- Funciona em full-duplex
 - Permite recepção e transmissão simultânea.
- A recepção e transmissão é feita pela leitura/escrita do registo **SBUF_n (double buffered)**

UART no 8051F388

receção: `char c = SBUF0;`

emissão: `SBUF0 = 'A';`

- A receção é “buffered”, ou seja existe um registo temporário no “shift register” de receção que guarda os bits do byte até este ter sido totalmente recebido.

UART no 8051F388

- Registos usados na programação da comunicação série:
(n indica o porto série a utilizar)
- **SCON_n**: Permite programar os modos de funcionamento e obter o estado da comunicação série.
- **SBUF_n**: Permite ler e escrever dados no porto série.
- **IE**: Permite controlar a geração da interrupção associada à UART
- Registos de programação do timer (baud rate generator)
- Mapear RxD e TxD através do Cross Bar: No TxD e RxD estão ligados a P0.4 e P0.5. Ver no **ConfigWizard2**

UART no 8051F388

- Para detalhes de configuração e descrição dos registos SCONn, flags Tin e RIn, interrupção, e configuração do baudrate, ver datasheet (pág 198/199) e/ou usar o ConfigWizard2!

SFR Definition 22.1. SCON0: Serial Port 0 Control								
Bit	7	6	5	4	3	2	1	0
Name	S0MODE	-	MCE0	REN0	TB80	RB80	TI0	RI0
Type	R/W	R	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	1	0	0	0	0	0	0

SFR Address = 0x98; SFR Page = All Pages; Bit-Addressable

Table 22.1. Timer Settings for Standard Baud Rates Using Internal Oscillator

	Target Baud Rate (bps)	Actual Baud Rate (bps)	Baud Rate Error	Oscillator Divide Factor	Timer Clock Source	SCA1-SCA0 (pre-scale select*)	T1M	Timer 1 Reload Value (hex)
SYSCLK = 12 MHz	230400	230769	0.16%	52	SYSCLK	XX	1	0xE6
	115200	115385	0.16%	104	SYSCLK	XX	1	0xCC
	57600	57692	0.16%	208	SYSCLK	XX	1	0x98
	28800	28846	0.16%	416	SYSCLK	XX	1	0x30
	14400	14423	0.16%	832	SYSCLK / 4	01	0	0x98
	9600	9615	0.16%	1248	SYSCLK / 4	01	0	0x64
	2400	2404	0.16%	4992	SYSCLK / 12	00	0	0x30
	1200	1202	0.16%	9984	SYSCLK / 48	10	0	0x98
SYSCLK = 24 MHz	230400	230769	0.16%	104	SYSCLK	XX	1	0xCC
	115200	115385	0.16%	208	SYSCLK	XX	1	0x98
	57600	57692	0.16%	416	SYSCLK	XX	1	0x30
	28800	28846	0.16%	832	SYSCLK / 4	01	0	0x98
	14400	14423	0.16%	1664	SYSCLK / 4	01	0	0x30
	9600	9615	0.16%	2496	SYSCLK / 12	00	0	0x98
	2400	2404	0.16%	9984	SYSCLK / 48	10	0	0x98
	1200	1202	0.16%	19968	SYSCLK / 48	10	0	0x30
SYSCLK = 48 MHz	230400	230769	0.16%	208	SYSCLK	XX	1	0x98
	115200	115385	0.16%	416	SYSCLK	XX	1	0x30
	57600	57692	0.16%	832	SYSCLK / 4	01	0	0x98
	28800	28846	0.16%	1664	SYSCLK / 4	01	0	0x30
	14400	14388	0.08%	3336	SYSCLK / 12	00	0	0x75
	9600	9615	0.16%	4992	SYSCLK / 12	00	0	0x30
	2400	2404	0.16%	19968	SYSCLK / 48	10	0	0x30

UART no 8051F388

- Enable UART0
 - TX & RX pins

Port I/O

Priority Crossbar | Ports 4 Input/Output

Pin I/O	P0	P1	P2	P3
TX0				
RX0				
SCK				
MISO				
MOSI				
NSS				
SDA				
SCL				
CP0				
CP0A				
CP1				
CP1A				
SYSCLK				
CEX0				
CEX1				
CEX2				
CEX3				
CEX4				
ECI				
T0				
T1				
TX1				
RX1				
SDA1				
SCL1				

Legend: ■ Assigned, ■ Skipped

D - Digital, A - Analog

O - Open Drain, P - Push-Pull

XBR0 = 0x01;
XBR1 = 0x40;

OK Cancel Reset

UART no 8051F388

- Enable UART0 RX

The screenshot shows the 'UARTs' configuration window with the 'UART0' tab selected. The settings are as follows:

- UART Mode:** ☒ 8 Bit UART, ☐ 9 Bit UART
- Receive Enable:** ☒ Enable UART0 Reception
- UART Interrupt:** UART0 Interrupt is Disabled. Button: [Configure UART Interrupts](#)
- UART Baud Rate:** 244 bps. Button: [Configure UART Baud Rate](#)
- Multiprocessor Communications Enable:**
 - 8 Bit UART - Checks for valid stop bit:
 - ☒ Logic level of stop bit ignored
 - ☐ RI0 will only be activated if stop bit is logic level 1
 - 9 Bit UART - MP Comm Enable:
 - ☒ Logic level of ninth bit is ignored
 - ☐ RI0 is set and an interrupt is generated only when the ninth bit is logic 1
- UART on the Crossbar:** UART0 is Enabled on the Crossbar. Button: [Configure Port I/O](#)

At the bottom, there is a text area containing the code: `SCON0 = 0x10;`

Buttons at the bottom: OK, Cancel, Reset.

UART no 8051F388

- Configure baudrate

- Timer1 8-bit counter auto-reload
 - `TMOD = 0x20;`
- Generate a baudrate of 115200 bps
 - `TH1 = 0x30;`
 - `CKCON = 0x08;`
- Set up flags...
 - `TCON = 0x40`
 - `TR1 = 1;`
 - Enable UART interrupt flag
 - `TIO = 1;`

Timers

Timer 1

Timer Mode

- ☐ 13 Bit Counter/Timer
- ☐ 16 Bit Counter/Timer
- ☒ 8 Bit Counter/Timer Auto-Reload
- ☐ Timer 1 Disabled

Prescaled Clock Inputs

- ☒ SYSCLK / 12
- ☐ SYSCLK / 4
- ☐ SYSCLK / 48
- ☐ External Oscillator / 8

Clock Source

- ☒ SYSCLK
- ☐ Prescaled Clock Input (Above)
- ☐ External T1

Configure External T1

Timer Interrupt

Timer 1 Interrupt is Disabled

Configure Timer Interrupts

Timer Control

- ☒ Enable Timer
- ☐ Gate Timer Depending on /INT1
 - ☒ /INT1 is Level Triggered
 - ☐ /INT1 is Edge Triggered

/INT1 is on P0.0

Configure /INT1 Ports

Timer Initial Value (Hex)

TH1 (High Byte - Reload Val in Mode 2): TL1 (Low Byte):

30 00

Bit Rate for SMBus0

76923 Hz

Change Bit Rate

Baud Rate for UART

115384 bps

Change Baud Rate

TCON = 0x40;
TMOD = 0x20;
CKCON = 0x08;
TH1 = 0x30;

OK Cancel Reset

Exemplos UART

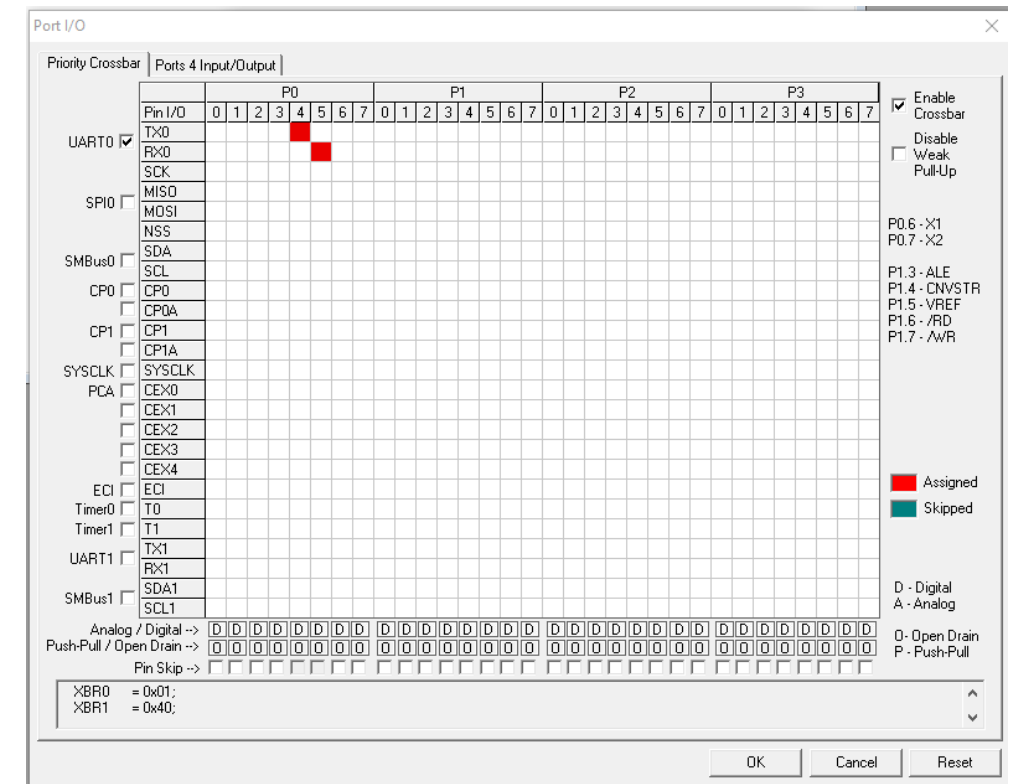
Hello World!

- Como enviar um byte pela porta série?
 - SCON0 = byte?
- E vários? Fazer um “Hello World! \n”?
 - ```
for (; ;){
 SCON = msg[]??
}
```
- Configurar e inicializar a UART?
- Podemos usar interrupções? TX e RX!

# Hello World!

- Configurar a UART0, 115200 bps usando timer 1, e enviar um byte!

```
24 /*****/
25 void system_init(void) {
26
27 PCA0MD = 0x00;
28 FLSEL = 0x90;
29 CLKSEL = 0x03;
30
31 //cross bar?
32 XBR0 = 0x01;
33 XBR1 = 0x40;
34 }
35
36
```



# Hello World!

- Inicialização

// 8 bit uart, enable RX, no 9<sup>th</sup> bit, ...

- `SCON0 = 0x10;`

// timer 1 Mode 2, 8-bit Counter for baudrate

- `TMOD = 0x20;`

// set timer 1 to use the system clock (48 MHz).

- `CKCON = 0x08;`

// timer 1 reload value (used for baudrate)

- `TH1 = ??;`

// run timer 1

- `TR1 = 1;`

// TX flag

- `TIO = 1;`

SFR Definition 22.1. SCON0: Serial Port 0 Control

| Bit   | 7      | 6 | 5    | 4    | 3    | 2    | 1   | 0   |
|-------|--------|---|------|------|------|------|-----|-----|
| Name  | S0MODE | - | MCE0 | REN0 | TB80 | RB80 | TIO | RI0 |
| Type  | R/W    | R | R/W  | R/W  | R/W  | R/W  | R/W | R/W |
| Reset | 0      | 1 | 0    | 0    | 0    | 0    | 0   | 0   |

SFR Address = 0x98; SFR Page = All Pages; Bit-Addressable

| Bit | Name   | Function                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | S0MODE | <b>Serial Port 0 Operation Mode.</b><br>Selects the UART0 Operation Mode.<br>0: 8-bit UART with Variable Baud Rate.<br>1: 9-bit UART with Variable Baud Rate.                                                                                                                                                                                                                                                                                                         |
| 6   | Unused | Read = 1b, Write = don't care.                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 5   | MCE0   | <b>Multiprocessor Communication Enable.</b><br>The function of this bit is dependent on the Serial Port 0 Operation Mode:<br><b>Mode 0: Checks for valid stop bit.</b><br>0: Logic level of stop bit is ignored.<br>1: RI0 will only be activated if stop bit is logic level 1.<br><b>Mode 1: Multiprocessor Communications Enable.</b><br>0: Logic level of ninth bit is ignored.<br>1: RI0 is set and an interrupt is generated only when the ninth bit is logic 1. |
| 4   | REN0   | <b>Receive Enable.</b><br>0: UART0 reception disabled.<br>1: UART0 reception enabled.                                                                                                                                                                                                                                                                                                                                                                                 |
| 3   | TB80   | <b>Ninth Transmission Bit.</b><br>The logic level of this bit will be sent as the ninth transmission bit in 9-bit UART Mode (Mode 1). Unused in 8-bit mode (Mode 0).                                                                                                                                                                                                                                                                                                  |
| 2   | RB80   | <b>Ninth Receive Bit.</b><br>RB80 is assigned the value of the STOP bit in Mode 0; it is assigned the value of the 9th data bit in Mode 1.                                                                                                                                                                                                                                                                                                                            |
| 1   | TIO    | <b>Transmit Interrupt Flag.</b><br>Set by hardware when a byte of data has been transmitted by UART0 (after the 8th bit in 8-bit UART Mode, or at the beginning of the STOP bit in 9-bit UART Mode). When the UART0 interrupt is enabled, setting this bit causes the CPU to vector to the UART0 interrupt service routine. This bit must be cleared manually by software.                                                                                            |
| 0   | RI0    | <b>Receive Interrupt Flag.</b><br>Set to 1 by hardware when a byte of data has been received by UART0 (set at the STOP bit sampling time). When the UART0 interrupt is enabled, setting this bit to 1 causes the CPU to vector to the UART0 interrupt service routine. This bit must be cleared manually by software.                                                                                                                                                 |

# Hello World!

- Baudrate 15200?

$T1\_Overflow\_Rate = 48\text{MHz} / 256 - 48$

$T1\_Overflow\_Rate = 230,769.2307$

Baudrate:  $230,769.2307 / 2 = 115385 ??$

Aprox. 15200!

TH1 = 0x30;

$$\text{UARTBaudRate} = \frac{1}{2} \times T1\_Overflow\_Rate$$

$$T1\_Overflow\_Rate = \frac{T1\_CLK}{256 - TH1}$$

Table 22.1. Timer Settings for Standard Baud Rates Using Internal Oscillator

|                 | Target Baud Rate (bps) | Actual Baud Rate (bps) | Baud Rate Error | Oscillator Divide Factor | Timer Clock Source | SCA1-SCA0 (pre-scale select*) | T1M | Timer 1 Reload Value (hex) |
|-----------------|------------------------|------------------------|-----------------|--------------------------|--------------------|-------------------------------|-----|----------------------------|
| SYSCLK = 12 MHz | 230400                 | 230769                 | 0.16%           | 52                       | SYSCLK             | XX                            | 1   | 0xE6                       |
|                 | 115200                 | 115385                 | 0.16%           | 104                      | SYSCLK             | XX                            | 1   | 0xCC                       |
|                 | 57600                  | 57692                  | 0.16%           | 208                      | SYSCLK             | XX                            | 1   | 0x98                       |
|                 | 28800                  | 28846                  | 0.16%           | 416                      | SYSCLK             | XX                            | 1   | 0x30                       |
|                 | 14400                  | 14423                  | 0.16%           | 832                      | SYSCLK / 4         | 01                            | 0   | 0x98                       |
|                 | 9600                   | 9615                   | 0.16%           | 1248                     | SYSCLK / 4         | 01                            | 0   | 0x64                       |
|                 | 2400                   | 2404                   | 0.16%           | 4992                     | SYSCLK / 12        | 00                            | 0   | 0x30                       |
|                 | 1200                   | 1202                   | 0.16%           | 9984                     | SYSCLK / 48        | 10                            | 0   | 0x98                       |
| SYSCLK = 24 MHz | 230400                 | 230769                 | 0.16%           | 104                      | SYSCLK             | XX                            | 1   | 0xCC                       |
|                 | 115200                 | 115385                 | 0.16%           | 208                      | SYSCLK             | XX                            | 1   | 0x98                       |
|                 | 57600                  | 57692                  | 0.16%           | 416                      | SYSCLK             | XX                            | 1   | 0x30                       |
|                 | 28800                  | 28846                  | 0.16%           | 832                      | SYSCLK / 4         | 01                            | 0   | 0x98                       |
|                 | 14400                  | 14423                  | 0.16%           | 1664                     | SYSCLK / 4         | 01                            | 0   | 0x30                       |
|                 | 9600                   | 9615                   | 0.16%           | 2496                     | SYSCLK / 12        | 00                            | 0   | 0x98                       |
|                 | 2400                   | 2404                   | 0.16%           | 9984                     | SYSCLK / 48        | 10                            | 0   | 0x98                       |
|                 | 1200                   | 1202                   | 0.16%           | 19968                    | SYSCLK / 48        | 10                            | 0   | 0x30                       |
| SYSCLK = 48 MHz | 230400                 | 230769                 | 0.16%           | 208                      | SYSCLK             | XX                            | 1   | 0x98                       |
|                 | 115200                 | 115385                 | 0.16%           | 416                      | SYSCLK             | XX                            | 1   | 0x30                       |
|                 | 57600                  | 57692                  | 0.16%           | 832                      | SYSCLK / 4         | 01                            | 0   | 0x98                       |
|                 | 28800                  | 28846                  | 0.16%           | 1664                     | SYSCLK / 4         | 01                            | 0   | 0x30                       |
|                 | 14400                  | 14388                  | 0.08%           | 3336                     | SYSCLK / 12        | 00                            | 0   | 0x75                       |
|                 | 9600                   | 9615                   | 0.16%           | 4992                     | SYSCLK / 12        | 00                            | 0   | 0x30                       |
|                 | 2400                   | 2404                   | 0.16%           | 19968                    | SYSCLK / 48        | 10                            | 0   | 0x30                       |

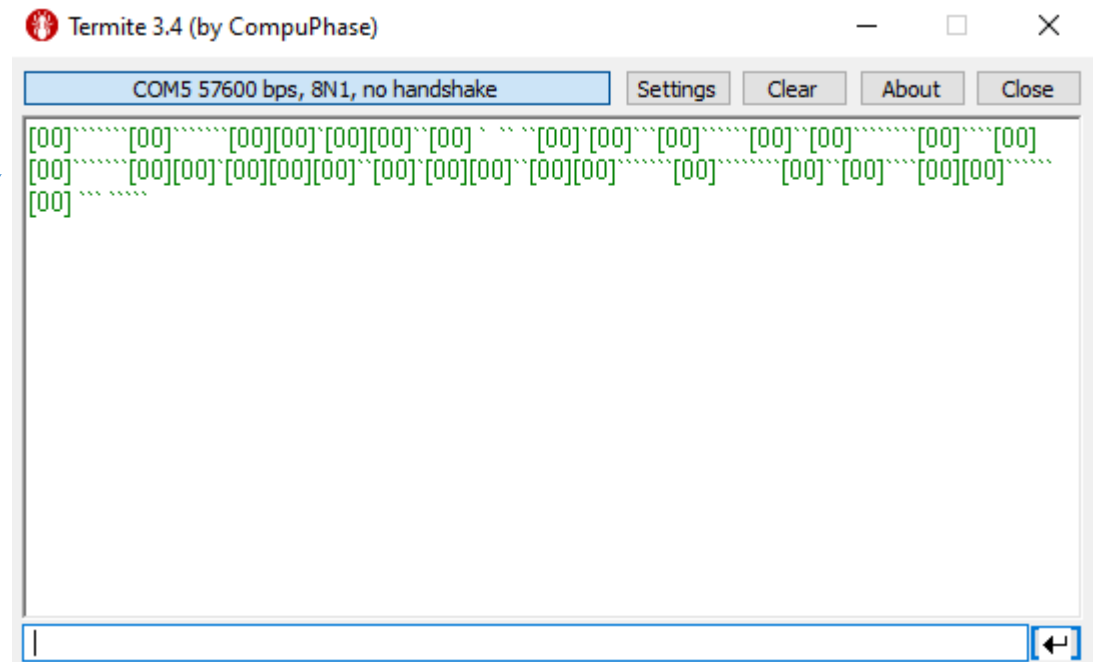


# Hello World!

- Library <stdio.h>
- `int putchar(int char);`

```
110 uart0_putc('A');
111 uart0_putc('\n');
112
```

```
153 /******
154 * putc using uart0
155 *****/
156 void uart0_putc(char c){
157
158 while(TIO == 0);
159 TIO = 0;
160 SBUF0 = c;
161
162 }
```

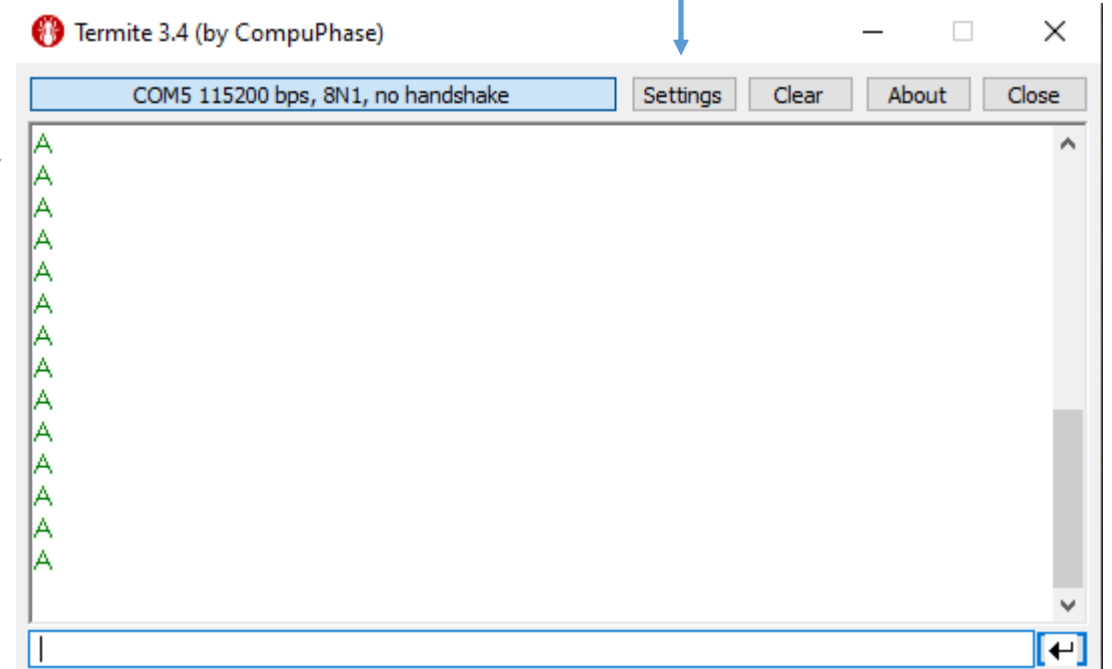


# Hello World!

- Library <stdio.h>
- `int putchar(int char);`

```
110 uart0_putc('A');
111 uart0_putc('\n');
112
```

```
153 /******
154 * putc using uart0
155 *****/
156 void uart0_putc(char c){
157
158 while(TIO == 0);
159 TIO = 0;
160 SBUF0 = c;
161
162 }
```

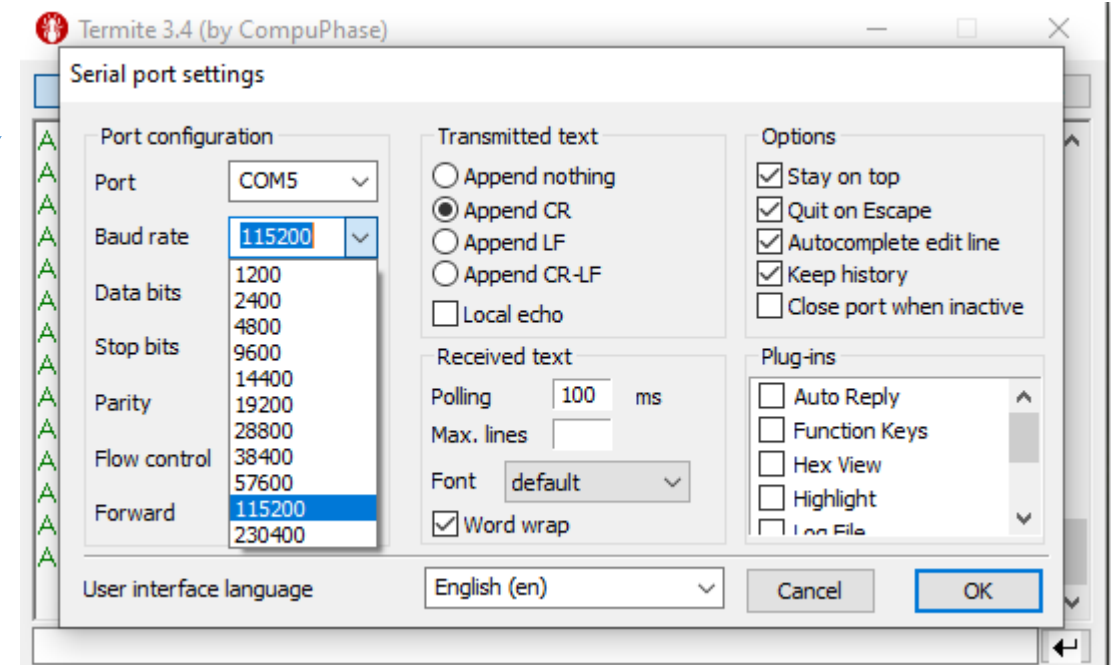


# Hello World!

- Library <stdio.h>
- `int putchar(int char);`

```
110 uart0_putc('A');
111 uart0_putc('\n');
112
```

```
153 /******
154 * putc using uart0
155 *****/
156 void uart0_putc(char c){
157
158 while(TIO == 0);
159 TIO = 0;
160 SBUF0 = c;
161
162 }
```

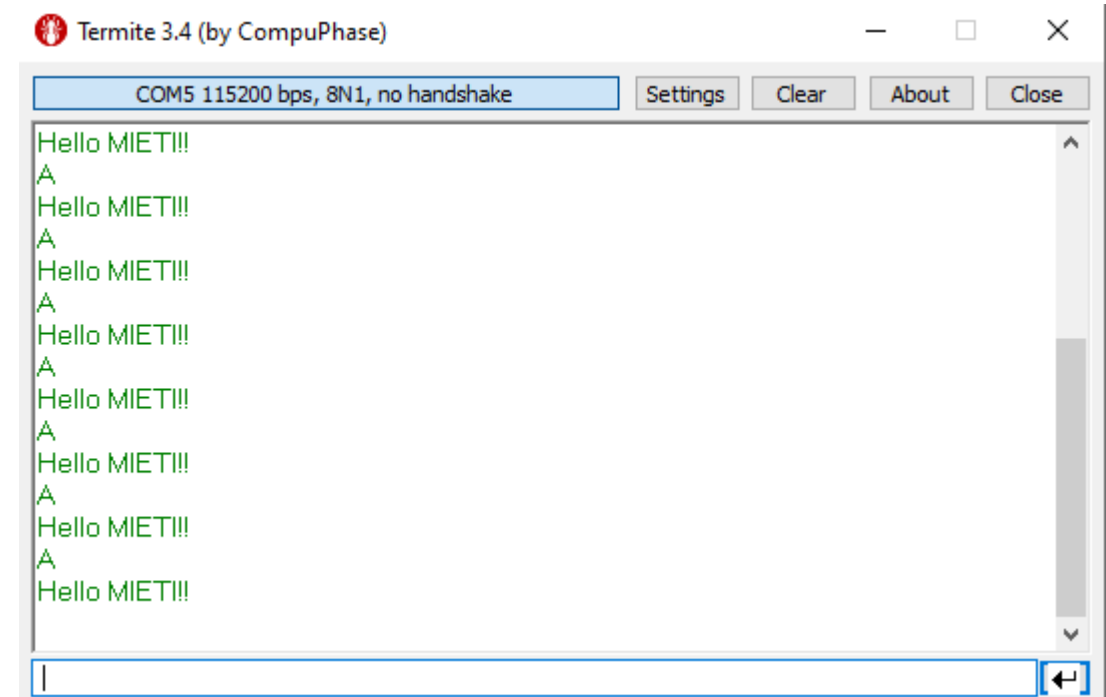


# Hello World!

- `print_u0(char * p_message)`

```
110 | uart0_putc('A');
111 | uart0_putc('\n');
112 |
113 | print_u0(message_u0);
```

```
132 | /*****
133 | * print using uart0
134 | *****/
135 | void print_u0(char* p_message) {
136 |
137 | unsigned char i = 0;
138 |
139 | while(p_message[i] != 0) {
140 |
141 | // Transmission flag
142 | while(TIO == 0);
143 |
144 | // UART write/read buffer (1 byte)
145 | SBUF0 = p_message[i];
146 |
147 | // Transmission flag control. set by hardware.
148 | TIO = 0;
149 | i++;
150 | }
151 | }
152 |
```



# Hello World!

---

- Funciona, mas estamos a usar polling!
- Soluções mais robustas utilizam rotinas associadas à interrupção da uart (activadas através das flags de TX e RX)
- Quando se quer enviar uma mensagem com vários bytes, estes são guardados num buffer de transmissão e enviados 1 a 1 dentro da rotina de interrupção TX.
  - Evita bloquear o processador à espera das flags TX/RX (polling). Enviar uma string de 16 bytes a 9600 bps demora mais de 16 ms...

# <stdio.h> printf() e scanf()

- Como funciona o printf()?

“The C library function `int printf(const char *format, ...)` sends **formatted** output to stdout.”

- Repare no seguinte programa:

```
for(ch = 75 ; ch <= 100; ch++) {
 printf("ASCII value = %d, Character = %c\n", ch , ch);
}
```

ASCII value = 75, Character = K

ASCII value = 76, Character = L

ASCII value = 77, Character = M

ASCII value = 78, Character = N

ASCII value = 79, Character = O

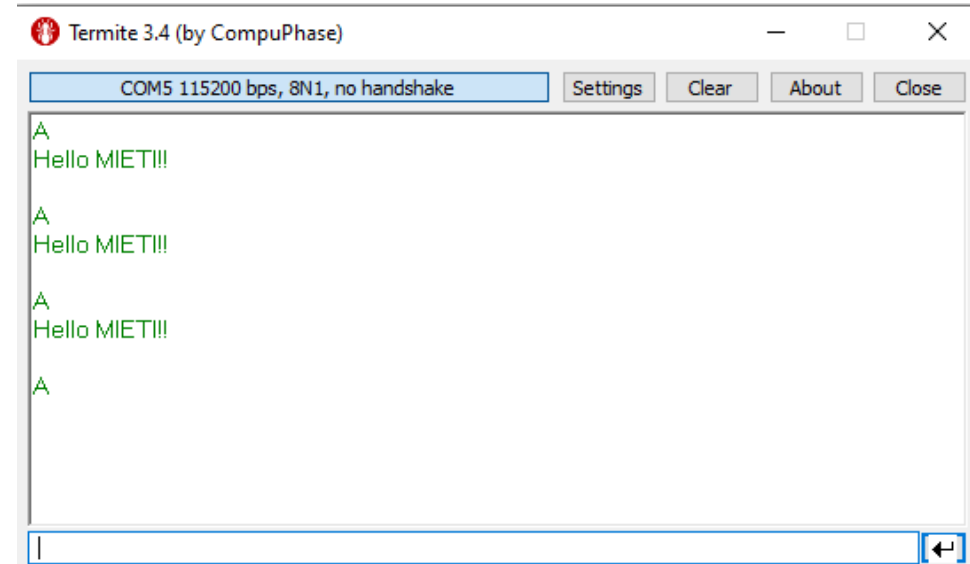
# <stdio.h> printf() e scanf()

- Como funciona o printf() no keil 8051?

## Note

- You must ensure that the argument type matches that of the format specification. You may use type casts to ensure that the proper type is passed to printf.
- This function is implementation-specific and is based on the operation of the `_getkey` and `putchar` functions. These functions, as provided in the standard library, read and write characters using the microcontroller's serial port. Custom functions may use other I/O devices.

```
110
111 uart0_putc('A');
112 uart0_putc('\n');
113
114 //print_u0(message_u0);
115
116 printf("Hello MIETI!!\r\n");
117
```



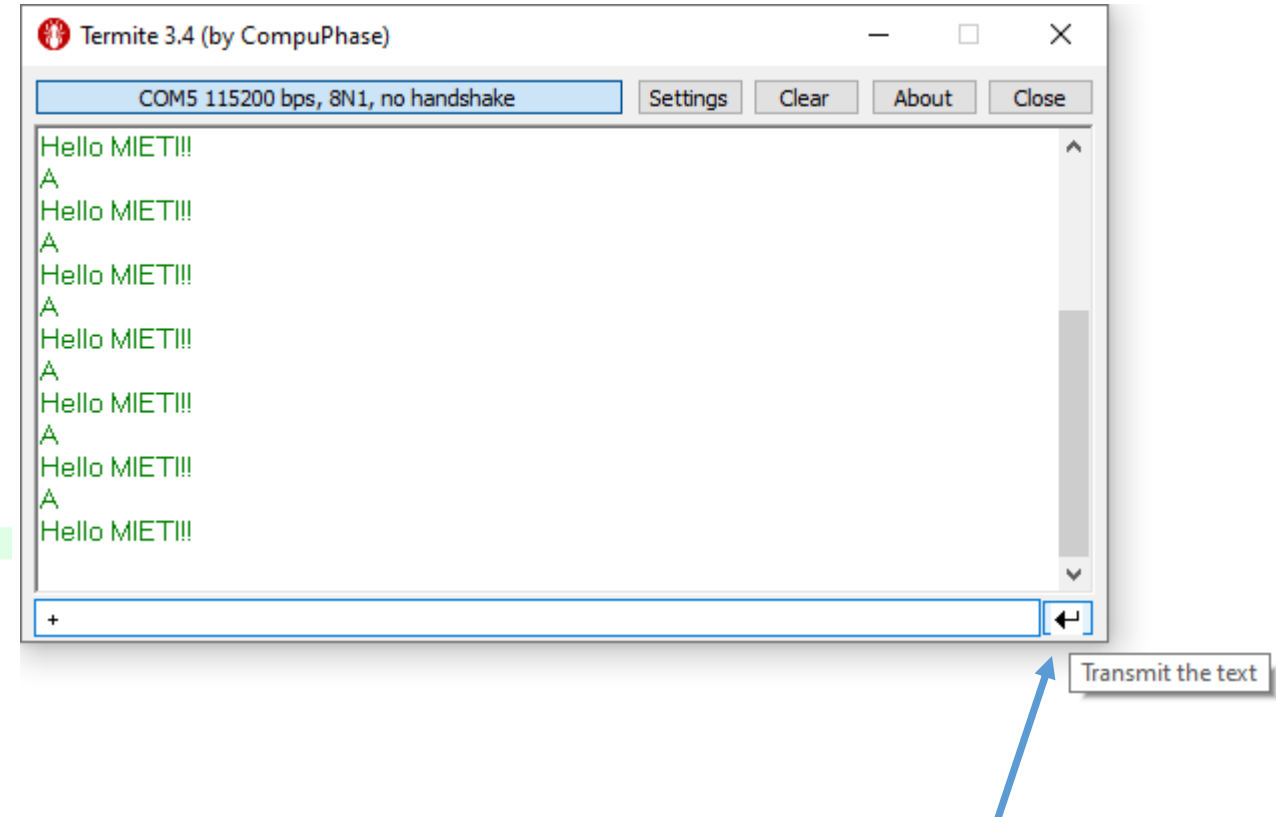
- Redefinimos o putchar, ou deixamos o keil usar a implementação nativa?

# <stdio.h> printf() e scanf()

- E para ler/receber um byte?

```
197 if(uart0_getc(&c)) {
198 switch(c) {
```

```
273 /*****
274 * getc using uart0
275 *****/
276 char uart0_getc(char* c) {
277
278 if(RIO != 0) {
279 RIO = 0;
280 *c = SBUF0;
281 return 1;
282 }
283 return 0;
284 }
```





# THANK YOU!

Questions?

[mr.gomes@dei.uminho.pt](mailto:mr.gomes@dei.uminho.pt)